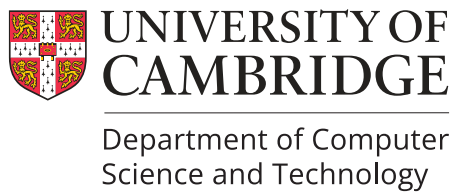


Categorical Imperative Programming

Type theory, refinement, and semantics for SSA

Jad-Elkhaleq Ghalayini

March 2026



This thesis is the result of my own work and includes nothing which is the outcome of work done in collaboration except as declared in the preface and specified in the text. It is not substantially the same as any work that has already been submitted, or is being concurrently submitted, for any degree, diploma or other qualification at the University of Cambridge or any other University or similar institution except as declared in the preface and specified in the text. It does not exceed the prescribed word limit for the relevant Degree Committee.

Contents

1. Introduction	4
2. S-Expressions	5
3. Language	5
4. State	6
5. Sequencing	7
6. Branching	8
7. Loops	9
8. Unstructured Control-Flow	10
Bibliography	12
1. Sample Appendix	13

1. Introduction

The fundamental question of this thesis is: *what does a computer program mean?*

We can approach this in two ways: from the bottom-up or the top-down.

The former tends to be more natural for computer scientists.

A computer is a machine for following instructions – a program is a sequence of instructions for it to follow.

In particular, a computer has an *instruction set* – a finite set of *operations* which it can perform, which read from and write to its *state* – that is, the current state of the registers, program counters, memory, interrupts, and all the other bits which make a modern CPU tick.

Which is a *lot*.

This thesis is about the *denotational semantics* of the *static single assignment* – or *SSA* – compiler *intermediate representation* – or *IR*.

I want to start by justifying this choice of topic.

- *Why* do we need a semantics for a *compiler IR*?

Because high-level languages have too many features to effectively study *or* to lower in one pass.

Because low-level languages are designed to be *executed* efficiently – making them difficult both to reason about *or* to lower in one pass.

- *Why* should we use *SSA* as our compiler *IR*?

Practically: most modern compilers use it.

Theoretically: it's right at the intersection of functional and imperative programming – letting us use it as a *thin waist* to develop the *isomorphism* between the two.

- *Why* should we give a *denotational* semantics?

Because we want to reason about programs *algebraically* – and to do so in a *compositional* manner.

We now want to give a formal account of the type-theoretic presentation of *SSA* we sketched in the introduction. In particular, for each of the intermediate representations we wish to study, we need to provide:

- A *grammar*, to enumerate the syntactically well-formed terms in our language – we've already provided this in the introduction for most of our intermediate representations.
- A *typing relation*, which allows us to determine whether a program is *semantically* well-formed.

In particular, we'll need a *type system* to construct our typing relation with respect to – later, we'll be able to relate different intermediate representations by giving them different typing relations over the same type system.

2. S-Expressions

TODO 1: This include should shift heading levels down by 1

3. Language

We'll start by giving a formal account of the simplest practical language – that of *first-order* arithmetic expressions and function calls, like $x + y$ and $a + (5 * b) + \sin(3)$.

Rather than complicate our language by introducing binary operators (like $x + y$), unary operators (like $-x$), and n -ary function applications (like $f(x, y, z)$), we can instead normalize everything to *S-expressions* – or *sexprs* – of the form $(f e_1 \dots e_n)$. We can then uniformly write:

- binary operations like $x + y$ as binary sexprs $(+ x y)$,
- unary operations $-x$ as unary sexprs $(- x)$,
- function applications $f(x, y, z)$ as n -ary sexprs $(f x y z)$.

In particular, the *operator* f always comes first, and there is no order-of-operations – all bracketing is explicit. Restricting ourselves to *first-order* sexprs means that we always require operators f to be atomic symbols drawn from a fixed set O – in particular, we don't support *partial application*, like $((\text{add } 2) 3)$, since that would require $f = (\text{add } 2)$ to be a compound expression.

Formally, we define our *syntax* as follows:

Definition 3.1 (First-order S-expressions): fixing

- a set of *variables* $x \in N$
- a set of *operations* $f \in O$
- a set of *constants* $c \in V$

we define the set of *first-order S-expressions* $e \in \text{STm}_N(O)$ to be given by the grammar

$$e ::= x \mid c \mid (f e^*)$$

TODO 2: Open-vs-closed programs – operational semantics of closed programs first

TODO 3: Operational semantics of S-expressions – partiality via division

TODO 4: *Observational equivalence* of S-expressions

TODO 5: Well-typed vs. ill-typed S-expressions

Given a set of *types* $A \in \mathcal{T}$, we'll define a *stack-typing relation* on O to be any relation

$$f : [A_1, \dots, A_m] \rightarrow [B_1, \dots, B_n]$$

where

- $A_1, \dots, A_m \in \mathcal{T}$ are types – m is called the *input arity* or *arity* of f .
- $B_1, \dots, B_n \in \mathcal{T}$ are types – n is called the *output arity* of f .

For now, we'll only consider operations with an output arity of 1.

We define a (*typing*) *context* $\Gamma \in \text{Ctx}_N(\mathcal{T})$ to be given by a list of *bindings*

$$x_1 : A_1, \dots, x_n : A_n$$

We can define a simple type system for S-expressions by giving a typing relation $\Gamma \vdash e : A$ that says “under the assumptions of Γ , the S-expression e has type A ”. The rules are:

TODO 6: Typing rules for constants – as distinct from nullary ops

$$\begin{array}{c} \text{var-head} \frac{}{\Gamma, x : A \vdash x : A} \qquad \text{var-tail} \frac{\Gamma \vdash y : B, x \neq y}{\Gamma, x : A \vdash y : B} \\[10pt] \text{app} \frac{f : [A_1, \dots, A_n] \rightarrow [B] \quad \forall i. \Gamma \vdash a_i : A_i}{\Gamma \vdash (f \ a_1 \ \dots \ a_n) : B} \end{array}$$

TODO 7: *Soundness* of typing: preservation of typing

TODO 8: *Completeness* of typing: in the presence of *simple types*

TODO 9: Church vs. Curry – we will only consider *well-typed* terms

TODO 10: Doesn't lose generality – untyping works in this framework due to multi-arity

TODO 11: *Substitution* and *observational equivalence* – typing rules for substitution – two open programs are equivalent iff equivalent under every substitution – this is usually where *metavariables* come in

TODO 12: *Purity* – required for substitution to preserve equivalence – typing rules for purity

TODO 13: Denotational semantics of S-expressions: partial functions

TODO 14: Denotational semantics of S-expressions: *soundness* and *completeness*

TODO 15: Equational theory of S-expressions: *purity* – as otherwise you can't *introduce* divisions

TODO 16: Equational theory of S-expressions: *soundness* and *completeness*

4. State

TODO 17: Operational semantics of S-expressions – *state*

TODO 18: *Observational equivalence* in the presence of state

TODO 19: Denotational semantics of S-expressions – the partial state monad

TODO 20: Denotational semantics of S-expressions: *soundness* and *completeness*

TODO 21: Equational theory of S-expressions: *purity*

TODO 22: Equational theory of S-expressions: *soundness* and *completeness*

5. Sequencing

6. Branching

7. Loops

8. Unstructured Control-Flow

Bibliography

1. Sample Appendix

SAMPLE TEXT